

CLI

# Permissions

Configure what the agent is allowed to do using permission tokens in your CLI configuration. Permissions are set in `~/.cursor/cli-config.json` (global) or `<project>/.cursor/cli.json` (project-specific).

## Permission types

### Shell commands

**Format:** `Shell(commandBase)`

Controls access to shell commands. The `commandBase` is the first token in the command line. Supports glob patterns and an optional `command:args` syntax for finer control.

| Example                    | Description                                                    |
|----------------------------|----------------------------------------------------------------|
| <code>Shell(ls)</code>     | Allow running <code>ls</code> commands                         |
| <code>Shell(git)</code>    | Allow any <code>git</code> subcommand                          |
| <code>Shell(npm)</code>    | Allow npm package manager commands                             |
| <code>Shell(curl:*)</code> | Allow <code>curl</code> with any arguments                     |
| <code>Shell(rm)</code>     | Deny destructive file removal (commonly in <code>deny</code> ) |

### File reads

**Format:** `Read(pathOrGlob)`

Controls read access to files and directories. Supports glob patterns.

| Example                        | Description                                        |
|--------------------------------|----------------------------------------------------|
| <code>Read(src/**/*.ts)</code> | Allow reading TypeScript files in <code>src</code> |
| <code>Read(**/*.md)</code>     | Allow reading markdown files anywhere              |
| <code>Read(.env*)</code>       | Deny reading environment files                     |
| <code>Read(/etc/passwd)</code> | Deny reading system files                          |

### File writes

**Format:** `Write(pathOrGlob)`

Controls write access to files and directories. Supports glob patterns. When using in print mode, `--force` is required to write files.

| Example                          | Description                                      |
|----------------------------------|--------------------------------------------------|
| <code>Write(src/**)</code>       | Allow writing to any file under <code>src</code> |
| <code>Write(package.json)</code> | Allow modifying package.json                     |
| <code>Write(**/*.key)</code>     | Deny writing private key files                   |

`Write(**/.env*)`

Deny writing environment files

## Web fetch

**Format:** `WebFetch(domainOrPattern)`

Controls which domains the agent can fetch when using the web fetch tool (e.g., to retrieve documentation or web pages). Without an allowlist entry, each fetch prompts for approval. Add domains to `allow` to auto-approve fetches from trusted sources.

| Example                                | Description                                                  |
|----------------------------------------|--------------------------------------------------------------|
| <code>WebFetch(docs.github.com)</code> | Allow fetches from <code>docs.github.com</code>              |
| <code>WebFetch(*.example.com)</code>   | Allow fetches from any subdomain of <code>example.com</code> |
| <code>WebFetch(*)</code>               | Allow fetches from any domain (use with caution)             |

### Domain pattern matching:

- `*` matches all domains
- `*.example.com` matches subdomains (e.g., `docs.example.com`, `api.example.com`)
- `example.com` matches that exact domain only

## MCP tools

**Format:** `Mcp(server:tool)`

Controls which MCP (Model Context Protocol) tools the agent can run. Use `server` (from `mcp.json`) and `tool` name, with `*` for wildcards.

| Example                     | Description                                 |
|-----------------------------|---------------------------------------------|
| <code>Mcp(datadog:*)</code> | Allow all tools from the Datadog MCP server |
| <code>Mcp(*:search)</code>  | Allow any server's <code>search</code> tool |
| <code>Mcp(*:*)</code>       | Allow all MCP tools (use with caution)      |

## Configuration

Add permissions to the `permissions` object in your CLI configuration file:

```
1  {
2    "permissions": {
3      "allow": [
4        "Shell(ls)",
5        "Shell(git)",
6        "Read(src/**/*.ts)",
7        "Write(package.json)",
8        "WebFetch(docs.github.com)",
9        "WebFetch(*.github.com)",
10       "Mcp(datadog:*)"
11     ],
12     "deny": [
13       "Shell(rm)",
14       "Read(.env*)",
15       "Write(**/*.key)",
16       "WebFetch(malicious-site.com)"
17     ]
18   }
19 }
```

## Pattern matching

- Glob patterns use `**`, `*`, and `?` wildcards
- Relative paths are scoped to the current workspace
- Absolute paths can target files outside the project
- Deny rules take precedence over allow rules
- Use `command:args` (e.g., `curl:*`) to match both command and arguments with globs



English ▾